

PROJETO DE ENGENHARIA DE SOFTWARE

Integração de Aplicação de Chat com IA (Next.js) em Plataforma WordPress

Empresa	Grupo Sul Brasil — DKN NonWoven TNT
Projeto	DKN Chat — Assistente Virtual Suh
Versão	1.0 — Proposta de Engenharia
Data	Junho de 2026
Status	Aprovado para implementação

1. Resumo Executivo

Este documento especifica a arquitetura técnica e o plano de implementação para integração da aplicação de chat com inteligência artificial do Grupo Sul Brasil DKN — denominada internamente "Suh" — no site institucional hospedado em WordPress (sulbrasiltnt.com.br).

A abordagem adotada segue o padrão de Arquitetura Desacoplada (Decoupled Architecture), isolando toda a inteligência computacional e processamento de dados na infraestrutura Next.js hospedada na Vercel, enquanto o WordPress atua exclusivamente como camada hospedeira da interface do usuário.

Esta é a abordagem padrão de mercado utilizada por empresas como Intercom, Drift e HubSpot para integração de chats inteligentes em plataformas de terceiros.

2. Contexto do Sistema Atual

O sistema DKN Chat já está em produção na Vercel com a seguinte arquitetura operacional:

- Assistente virtual com persona "Suh" — responde dúvidas técnicas e comerciais sobre TNT
- Roteamento inteligente de intenções via agente Facilitador (classifica a mensagem antes de responder)
- Três agentes especializados: Comercial (Suh), RH e Compras — cada um com prompt e modelo configuráveis
- Encaminhamento para representantes regionais via WhatsApp com detecção de estado (UF) do cliente
- Histórico de conversas salvo em Google Sheets com métricas de custo por sessão
- Painel de monitoramento em /painel para avaliação manual de conversas (Sucesso/Falha)
- 5 camadas de segurança: rate limiting, validação de mensagem, proteção contra prompt injection, anti-alucinação e timeout

3. Stack Tecnológica

Camada	Tecnologia	Observação
Framework	Next.js 16.2.6	App Router, API Routes serverless
Linguagem	TypeScript	Tipagem estática em todo o projeto
IA — Agentes	Gemini 2.5 Flash	Endpoint /v1beta/, 3 tentativas com backoff
IA — Facilitador	Gemini 3.1 Flash Lite	Modelo GA estável, classifica intenções
Armazenamento	Google Sheets	Histórico de conversas, migração para PostgreSQL prevista

Camada	Tecnologia	Observação
Hospedagem	Vercel (Hobby)	Serverless, maxDuration 30s para /api/chat
Build widget	Vite (a implementar)	Compila ChatWidget.tsx como bundle IIFE standalone
Integração WP	Web Component + Shadow DOM	Isolamento de CSS, injeção via <script defer>

4. Arquitetura Proposta — Divisão de Responsabilidades

4.1 Princípio Fundamental

O sistema operará com divisão estrita de escopo entre as duas plataformas. Nenhum processamento de inteligência artificial, gestão de sessão ou acesso a APIs externas ocorrerá no ambiente WordPress.

Camada Next.js / Vercel (Backend)	Camada WordPress (Apresentação)
Chaves de API e variáveis de ambiente protegidas	Injetar e renderizar o widget visual
Controle de sessão e histórico de mensagens	Nenhum processamento de IA
Lógica dos agentes (Suh, RH, Compras)	Nenhuma query ao banco de dados relacionada ao chat
Retry, backoff e fallback para atendimento humano	Zero impacto no desempenho do site principal
Armazenamento de logs e métricas de custo	Uma linha de código no footer

4.2 Fluxo de Comunicação

O fluxo completo desde o acesso do usuário até a resposta da IA segue a sequência abaixo:

- Usuário acessa sulbrasilnt.com.br — WordPress carrega normalmente
- Browser detecta o <script defer> no footer e baixa chat-widget.js da Vercel (~50–150 KB)
- Web Component registra o elemento <dkn-chat> e renderiza o botão flutuante do chat
- Usuário abre o chat e envia uma mensagem
- Widget envia POST direto para https://project-0qgvo.vercel.app/api/chat — WordPress nunca vê a mensagem
- Facilitador classifica a intenção (Gemini 3.1 Flash Lite)
- Agente correto processa e responde (Gemini 2.5 Flash)
- Resposta volta ao widget — conversa salva no Google Sheets
- Widget exibe a resposta ao usuário com botões de ação (WhatsApp, Distribuidor etc.)

5. Plano de Implementação — Etapas de Refatoração

A refatoração necessária para transformar o ChatWidget.tsx atual em um Web Component distribuível é composta por seis etapas sequenciais. O sistema em produção não sofre nenhuma interrupção durante este processo.

#	Etapas	Descrição	Estimativa
1	Configurar Vite	Criar vite.config.ts para compilar ChatWidget.tsx como bundle standalone (library mode). Definir entry point, formato IIFE e nome global do componente.	2–4h
2	Isolar o componente	Extrair ChatWidget.tsx de qualquer dependência exclusiva do Next.js (remoção de imports de next/image, next/link etc.). Garantir que o componente funcione standalone.	2–4h
3	Web Component wrapper	Criar chat-widget.ts que registra o componente React como customElements.define('dkn-chat', ...) usando Shadow DOM para isolamento de CSS.	3–5h
4	Build e hospedagem	Rodar vite build. O arquivo chat-widget.js é gerado em /public. A Vercel o serve automaticamente como arquivo estático em /chat-widget.js.	1h
5	Injeção no WordPress	No Elementor (ou functions.php), adicionar uma linha no footer: <script src='URL/chat-widget.js' defer></script> + <dkn-chat></dkn-chat>.	30min
6	Testes de integração	Validar isolamento de CSS (Shadow DOM), chamadas à API /api/chat, comportamento em mobile, e ausência de conflitos com o tema WordPress.	2–4h

Estimativa total: 10 a 18 horas de desenvolvimento. O sistema em produção (Vercel) permanece 100% operacional durante toda a refatoração — não há risco de downtime.

6. Análise Comparativa de Desempenho

Comparação direta entre a abordagem proposta e a alternativa de plugins WordPress nativos:

Métrica	Abordagem Proposta (Next.js)	Plugins WordPress
Carga de CPU no servidor WP	Zero — processamento na Vercel	Alta — cada request consome PHP
Impacto no Core Web Vitals	Nulo — script assíncrono (defer)	Negativo — pode bloquear renderização
Banco de dados WordPress	Inexistente — logs na Vercel/Sheets	Alto — histórico congestiona tabelas SQL
Segurança das chaves de API	Máxima — variáveis server-side Node.js	Moderada a baixa — risco de exposição
Manutenibilidade	Alta — deploy único atualiza tudo	Baixa — dependência de plugins de terceiros
Escalabilidade	Horizontal via Vercel serverless	Limitada pela infraestrutura do host WP

7. Benefícios Estratégicos

7.1 Independência Tecnológica

Caso o Grupo Sul Brasil decida migrar o site principal do WordPress para qualquer outra plataforma no futuro, o sistema de chat permanecerá intacto e operacional. A única alteração necessária seria apontar o script de injeção na nova plataforma.

7.2 Segurança de Dados

O desacoplamento garante que eventuais vulnerabilidades no WordPress (plugins desatualizados, ataques à instalação) não comprometam os logs de conversas, chaves de API do Gemini ou a lógica proprietária dos agentes.

7.3 Desempenho do Site Principal

O WordPress continua leve e veloz. O carregamento assíncrono do widget (atributo defer) garante que o script da Suh nunca bloqueie a renderização das páginas do site, preservando os Core Web Vitals e o SEO.

7.4 Escalabilidade

A infraestrutura Vercel escala horizontalmente de forma automática conforme o volume de conversas aumenta, sem qualquer impacto na infraestrutura do WordPress.

8. Riscos e Mitigações

Risco	Impacto	Mitigação
Gemini 2.5 Flash em preview	Médio	Retry com backoff exponencial (3 tentativas). Falha persistente encaminha para atendimento humano via WhatsApp.
Google Sheets como banco de dados	Baixo	Funcional para fase de testes. Migração para PostgreSQL prevista quando volume aumentar.
Conflito de CSS no WordPress	Médio	Mitigado pelo Shadow DOM do Web Component. CSS do tema WP não penetra no shadow root.
Chave GOOGLE_PRIVATE_KEY na Vercel	Baixo	Deve ser salva com aspas duplas. Redeploy que resetar a variável quebra integração com Sheets.
Logs da Vercel somem após 1h (Hobby)	Médio	Erros invisíveis após 1 hora. Implementar monitoramento persistente após aprovação do MVP.

9. Próximos Passos — Roadmap

Fase	Item	Descrição	Prazo Est.
MVP	Widget embed	Configuração Vite + extração do ChatWidget + injeção no WP via iframe (validação rápida)	1–2 dias
v1.0	Web Component	Refatoração completa com Shadow DOM — produto final de alto nível	2–3 dias
v1.1	Agente Financeiro	Implementação após recebimento dos dados do departamento financeiro	A definir
v1.2	Representantes completos	Lista completa (~20 representantes) com roteamento preciso por estado	A definir
v2.0	Migração para PostgreSQL	Substituição do Google Sheets por banco relacional quando o volume justificar	A definir

10. Mapa Visual da Arquitetura de Integração

O diagrama abaixo ilustra o fluxo completo de integração, desde o código-fonte até o usuário final:



↓ npx vercel --prod

[Etapa 3] Vercel hospeda: chat-widget.js (público) + /api/chat (privado, chaves protegidas)

↓ uma linha no footer do WordPress

[Etapa 4] WordPress carrega: `<script src='URL/chat-widget.js' defer></script>`

↓

[Etapa 5] Usuário vê o chat da Suh no site. Mensagens vão direto à Vercel — WordPress não processa nada.

Referência visual interativa completa disponível em: <https://project-0qgvo.vercel.app>

11. Conclusão

A arquitetura desacoplada proposta neste documento representa o padrão de mercado para integração de sistemas de chat com inteligência artificial em plataformas CMS. A abordagem garante máxima segurança das chaves de API, zero impacto no desempenho do site WordPress, independência tecnológica de longo prazo e escalabilidade horizontal automática.

O sistema DKN Chat já está em produção e operacional. O trabalho remanescente para atingir o estado descrito neste documento é de 10 a 18 horas de desenvolvimento, com risco zero de downtime para o site principal.

Argumento técnico síntese: *"O WordPress atua exclusivamente como camada de apresentação. Todo o processamento de IA, gestão de sessão e segurança de chaves permanece isolado na infraestrutura Next.js na Vercel — arquitetura desacoplada padrão de mercado."*